

MA-StatDig - Laboratoire 4-3

HEIG-VD, Prof. M. Tognolini

March 23, 2026

1 Exercice 4.3 : Enregistrements de données finies

1.1 Méthodes d'Autocorrélation et de Covariance

(M.Tognolini HEIG-VD 2026 MA-StatDig)

Ce laboratoire explore la modélisation “All-Pole” de second ordre pour un signal sinusoïdal. Nous comparons les résultats théoriques avec deux approches pratiques sur des records de données finis :

1. **La méthode d'Autocorrélation (ACM)** 2. **La méthode de Covariance (COVM)**

Le modèle visé est :

$$H(z) = \frac{b(0)}{1 + a(1)z^{-1} + a(2)z^{-2}}$$

Les équations normales à résoudre sont :

$$\begin{bmatrix} r_x(0) & r_x(1) \\ r_x(1) & r_x(0) \end{bmatrix} \begin{bmatrix} a(1) \\ a(2) \end{bmatrix} = - \begin{bmatrix} r_x(1) \\ r_x(2) \end{bmatrix}$$

Try several number of samples N , first a multiple of the period length (ex 40) and also 38 or 42 or 55 the result will be very different to the theory.

1.1.1 Importation des librairies et Déclaration des fonctions

```
[4]: import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import toeplitz, inv, solve
import scipy.signal as signal
```

1.2 4.1.1. Modèle Théorique

L'autocorrélation théorique pour une sinusoïde est donnée par :

$$r_x(k) = \frac{1}{2}A^2 \cos(k\omega_0)$$

Nous calculons les coefficients a_p théoriques à partir de ces valeurs.

```
# definition du signal
NO = 20
N= 55
```

```

n = np.arange(N)
A=1
x=A * np.sin(2 * np.pi * n / N0)

# Autocorrélation théorique
rxth = 0.5 * np.cos(np.arange(3) * 2 * np.pi / N0)
Rxth = toeplitz(rxth[:2])
rx_vec_th = rxth[1:3]

# Calcul des coefficients théoriques
apth_tail = -solve(Rxth, rx_vec_th)
apth = np.concatenate(([1.0], apth_tail))

```

```

[5]: # definition du signal
N0 = 20
N= 55
n = np.arange(N)
A=1
x=A * np.sin(2 * np.pi * n / N0)

# Autocorrélation théorique
rxth = 0.5 * np.cos(np.arange(3) * 2 * np.pi / N0)
Rxth = toeplitz(rxth[:2])
rx_vec_th = rxth[1:3]

# Calcul des coefficients théoriques
apth_tail = -solve(Rxth, rx_vec_th)
apth = np.concatenate(([1.0], apth_tail))

# Pôles théoriques (doivent être sur le cercle unité, norme = 1)
pth = np.roots(apth)
print(f"Coefficients théoriques ap : {apth}")
print(f"Norme des pôles : {np.abs(pth)}")

# Approx error
#Ep = rxth[0] + apth(1)*rxth(1) + apth(2)*rxth(2)
Ep = np.dot(rxth,apth)
print("Ep", Ep)
#pth = roots(apth)

# poles of the filter

```

```

pth = np.roots(apth)
print("pth", pth)
# magnitude of the poles
magpth = np.abs(pth)
print("magpth", magpth)

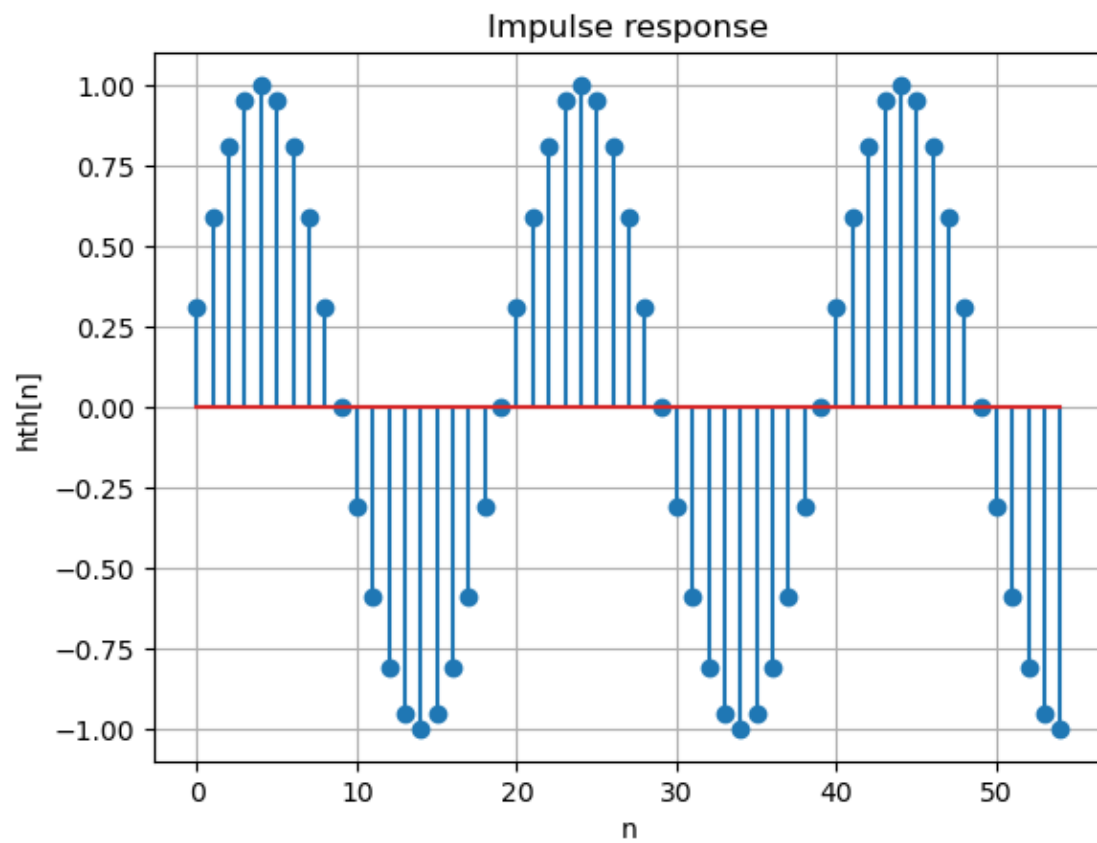
b0 = np.sin(2*np.pi/N0)
# transfer function H(z)
bth = np.array([b0,0,0])
# impulse response
dn = np.eye(1, N)[0]
hth = signal.lfilter(bth, apth, dn)
nth = n

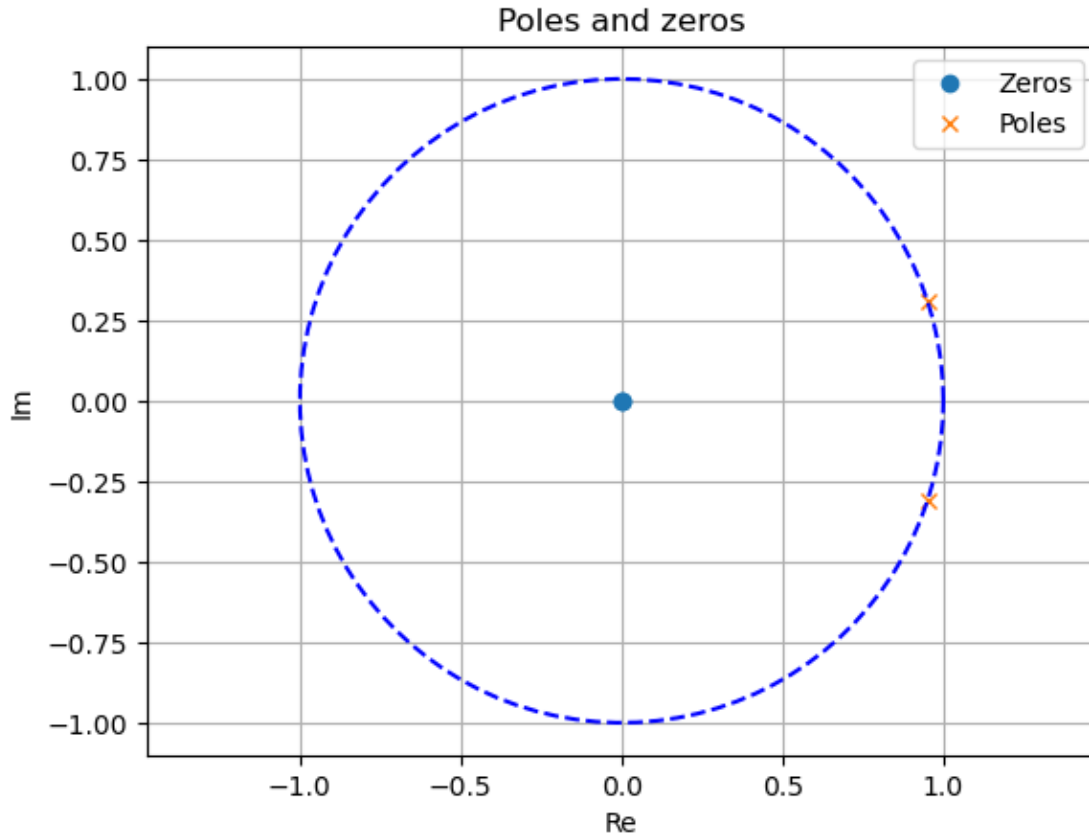
# plot
plt.figure(1)
plt.stem(nth, hth)
plt.title('Impulse response')
plt.xlabel('n')
plt.ylabel('hth[n]')
plt.grid()
plt.show()

# poles and zeros plot with unity circle
plt.figure(2)
zth, pth, kth = signal.tf2zpk(bth, apth)
plt.plot(np.real(zth), np.imag(zth), 'o', label='Zeros')
plt.plot(np.real(pth), np.imag(pth), 'x', label='Poles')
plt.plot(np.cos(np.linspace(0, 2*np.pi, 100)), np.sin(np.linspace(0, 2*np.pi, 100)), '--b')
plt.axis('equal')
plt.title('Poles and zeros')
plt.xlabel('Re')
plt.ylabel('Im')
plt.grid()
plt.legend()
plt.show()

```

Coefficients théoriques ap : [1. -1.90211303 1.]
 Norme des pôles : [1. 1.]
 Ep 2.4634150837260204e-16
 pth [0.95105652+0.30901699j 0.95105652-0.30901699j]
 magpth [1. 1.]





1.3 2. Méthode d'Autocorrélation (ACM)

La méthode d'autocorrélation estime $r_x(k)$ en supposant que le signal est nul en dehors de la fenêtre de N points :

$$\hat{r}_x(k) = \frac{1}{N} \sum_{n=k}^{N-1} x(n)x(n-k) \quad ; \quad k \geq 0$$

Cela introduit un biais qui tend à déplacer les pôles vers l'intérieur du cercle unité, créant une réponse impulsionnelle de type "sinusoïde amortie".

Creez une fonction `acm` qui prend en entrée un signal `x` et le nombre de pôles `p` et retourne les coefficients `ap`, `bq0` et l'erreur de prédiction.

```
def acm(x, p):
    N = len(x)
    # Construction de la matrice de données avec padding de zéros
    # Similaire à convmtx(x, p+1)
    X = toeplitz(np.concatenate([x, np.zeros(p)]), np.zeros(p+1))
    Xp = X[:N+p-1, :p]
    #print("Xp", Xp)
    # Matrice Rx et vecteur rx
```

```

Rx = (Xp.T @ Xp) / N
print("Rx", Rx)
rx_vec = (Xp.T @ X[1:N+p+1,0].reshape(-1, 1)) / N
print("rx_vec", rx_vec)

# Plus robuste : utiliser l'autocorrélation directe
r = np.correlate(x, x, mode='full')[N-1 : N+p] / N
R_mat = toeplitz(r[:p])
r_vec = r[1:p+1]

ap_tail = -solve(R_mat, r_vec)
ap = np.concatenate(([1.0], ap_tail))

# Calcul du gain bq0
err = r[0] + np.dot(ap_tail, r[1:p+1])
bq0 = np.sqrt(np.abs(err))

return ap, bq0, err

```

```

[6]: def acm(x, p):
    N = len(x)
    # Construction de la matrice de données avec padding de zéros
    # Similaire à convmtx(x, p+1)
    X = toeplitz(np.concatenate([x, np.zeros(p)]), np.zeros(p+1))
    Xp = X[:N+p-1, :p]
    #print("Xp", Xp)
    # Matrice Rx et vecteur rx
    Rx = (Xp.T @ Xp) / N
    print("Rx", Rx)
    rx_vec = (Xp.T @ X[1:N+p+1,0].reshape(-1, 1)) / N
    print("rx_vec", rx_vec)

    # Plus robuste : utiliser l'autocorrélation directe
    r = np.correlate(x, x, mode='full')[N-1 : N+p] / N
    R_mat = toeplitz(r[:p])
    r_vec = r[1:p+1]

    ap_tail = -solve(R_mat, r_vec)
    ap = np.concatenate(([1.0], ap_tail))

    # Calcul du gain bq0
    err = r[0] + np.dot(ap_tail, r[1:p+1])
    bq0 = np.sqrt(np.abs(err))

    return ap, bq0, err

print("Signal x:", x)

```

```

ap_acm, bq0_acm, err_acm = acm(x, 2)
poles_acm = np.roots(ap_acm)
bq0_acm = np.array([bq0_acm,0])
print(f"Norme des pôles (ACM) : {np.abs(poles_acm)}")
print(f"Coefficients ACM ap : {ap_acm}")
print(f"Gain ACM bq0 : {bq0_acm}")
print(f"Erreur de prédiction (ACM) : {err_acm}")

# impulse response
hacm = signal.lfilter(bq0_acm, ap_acm, dn)

# plot
plt.figure(1)
plt.plot(n, hacm, 'bo-', label = 'hacm(n)')
plt.plot(n, x, '')
plt.title('Impulse response with autocorrelation method')
plt.xlabel('n')
plt.ylabel('h_acm[n]')
plt.grid()
plt.show()

# poles and zeros plot with unity circle
z_acm, p_acm, k_acm = signal.tf2zpk([bq0_acm], ap_acm)

magp_acm = np.abs(p_acm)
print("magp", magp_acm)

plt.figure(3)
plt.plot(np.real(z_acm), np.imag(z_acm), 'o', label='Zeros')
plt.plot(np.real(p_acm), np.imag(p_acm), 'x', label='Poles')
plt.plot(np.cos(np.linspace(0, 2*np.pi, 100)), np.sin(np.linspace(0, 2*np.pi, 100)), '--b')
plt.axis('equal')
plt.title('Poles and zeros of the estimated filter autocorrelation method')
plt.xlabel('Re')
plt.ylabel('Im')
plt.grid()
plt.legend()
plt.show()

```

```

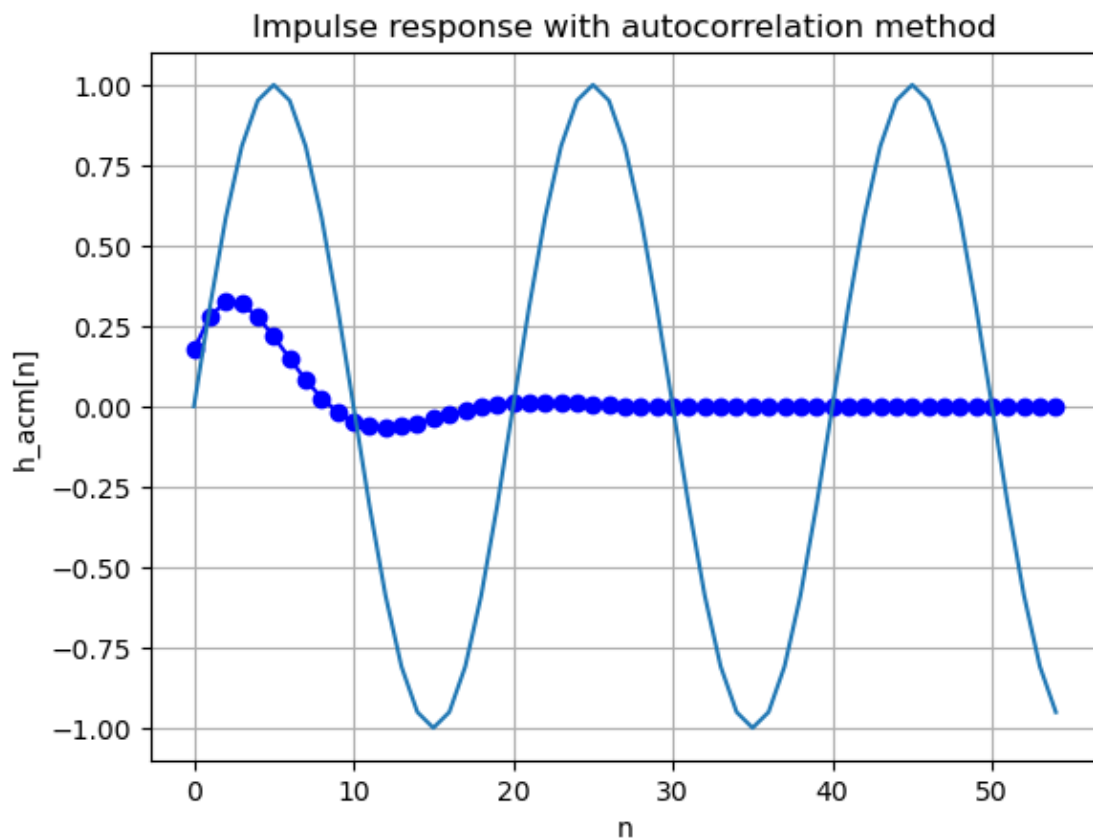
Signal x: [ 0.00000000e+00  3.09016994e-01  5.87785252e-01  8.09016994e-01
 9.51056516e-01  1.00000000e+00  9.51056516e-01  8.09016994e-01
 5.87785252e-01  3.09016994e-01  1.22464680e-16 -3.09016994e-01
-5.87785252e-01 -8.09016994e-01 -9.51056516e-01 -1.00000000e+00
-9.51056516e-01 -8.09016994e-01 -5.87785252e-01 -3.09016994e-01

```

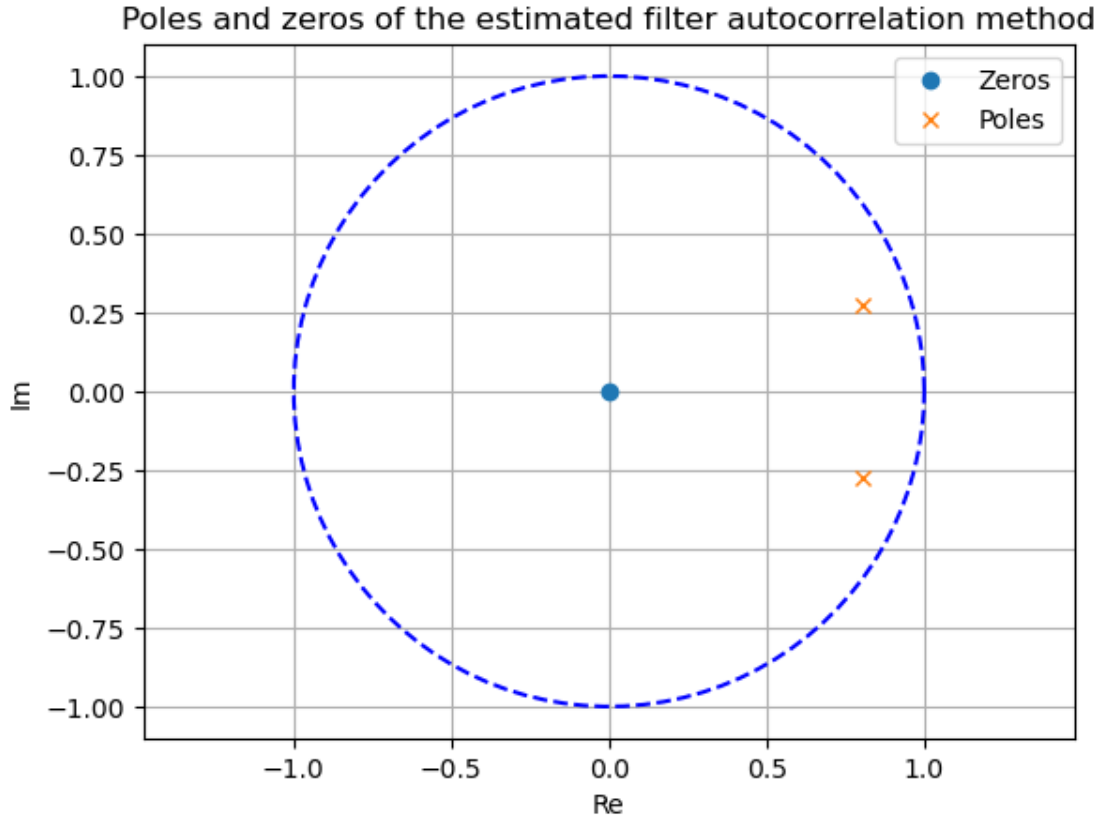
```

-2.44929360e-16  3.09016994e-01  5.87785252e-01  8.09016994e-01
 9.51056516e-01  1.00000000e+00  9.51056516e-01  8.09016994e-01
 5.87785252e-01  3.09016994e-01  3.67394040e-16 -3.09016994e-01
-5.87785252e-01 -8.09016994e-01 -9.51056516e-01 -1.00000000e+00
-9.51056516e-01 -8.09016994e-01 -5.87785252e-01 -3.09016994e-01
-4.89858720e-16  3.09016994e-01  5.87785252e-01  8.09016994e-01
 9.51056516e-01  1.00000000e+00  9.51056516e-01  8.09016994e-01
 5.87785252e-01  3.09016994e-01  6.12323400e-16 -3.09016994e-01
-5.87785252e-01 -8.09016994e-01 -9.51056516e-01]
Rx [[0.49090909 0.45823632]
     [0.45823632 0.49090909]]
rx_vec [[0.45823632]
         [0.3824444 ]]
Norme des pôles (ACM) : [0.84675963 0.84675963]
Coefficients ACM ap : [ 1.          -1.60272571  0.71700187]
Gain ACM bq0 : [0.17520076 0.          ]
Erreur de prédiction (ACM) : 0.030695305742758028

```



```
magp [0.84675963 0.84675963]
```



1.4 3. Méthode de Covariance (COVM)

La méthode de covariance n'effectue pas de padding de zéros. Elle estime la corrélation uniquement sur les données disponibles :

$$\hat{r}_{xcov}(k, l) = \frac{1}{N-p} \sum_{n=p}^{N-1} x(n-l)x(n-k)$$

Cette méthode est généralement beaucoup plus précise pour estimer des signaux purement sinusoïdaux sur des records courts.

Créez une fonction `covm` qui prend en entrée un signal `x` et le nombre de pôles `p` et retourne les coefficients `ap`, `bq0` et l'erreur de prédiction. “python def covm(x, p): N = len(x) # Matrice des données sans padding X = toeplitz(x[p-1:N-1], x[p-1:-1]) # Cible x_target = x[p:N]

```
# Résolution par moindres carrés
ap_tail = -solve(X.T @ X, X.T @ x_target)
ap = np.concatenate(([1.0], ap_tail))

# Erreur et Gain
err = np.mean((x_target + X @ ap_tail)**2)
```

```
bq0 = np.sqrt(err)

return ap, bq0, err
```

```
[7]: def covm(x, p):
    N = len(x)
    # Matrice des données sans padding
    X = toeplitz(x[p-1:N-1], x[p-1::-1])
    # Cible
    x_target = x[p:N]

    # Résolution par moindres carrés
    ap_tail = -solve(X.T @ X, X.T @ x_target)
    ap = np.concatenate([1.0], ap_tail)

    # Erreur et Gain
    err = np.mean((x_target - X @ ap_tail)**2)
    bq0 = np.sqrt(err)

    return ap, bq0, err

ap_cov, bq0_cov, err_cov = covm(x, 2)
bq0_cov = np.array([A * np.sin(2*np.pi/N0), 0])
poles_cov = np.roots(ap_cov)
print(f"Coefficients COVM ap : {ap_cov}")
print(f"Gain COVM bq0 : {bq0_cov}")
print(f"Erreur de prédiction (COVM) : {err_cov}")
print(f"Norme des pôles (COVM) : {np.abs(poles_cov)}")
```

```
Coefficients COVM ap : [ 1.          -1.90211303  1.          ]
Gain COVM bq0 : [0.30901699  0.          ]
Erreur de prédiction (COVM) : 6.294808188154608e-30
Norme des pôles (COVM) : [1.  1.]
```

1.5 4. Comparaison Graphique

Nous comparons la position des pôles dans le plan complexe et les réponses impulsionnelles générées par chaque méthode.

```
[8]: # Plan Complexe
plt.figure(figsize=(7, 7))
theta = np.linspace(0, 2*np.pi, 100)
plt.plot(np.cos(theta), np.sin(theta), 'k--', alpha=0.5, label="Cercle unité")
plt.scatter(np.real(pth), np.imag(pth), marker='o', edgecolors='b',
    ↳facecolors='none', s=100, label="Théorie")
plt.scatter(np.real(poles_acm), np.imag(poles_acm), marker='x', c='r', s=100,
    ↳label="Autocorr (ACM)")
```

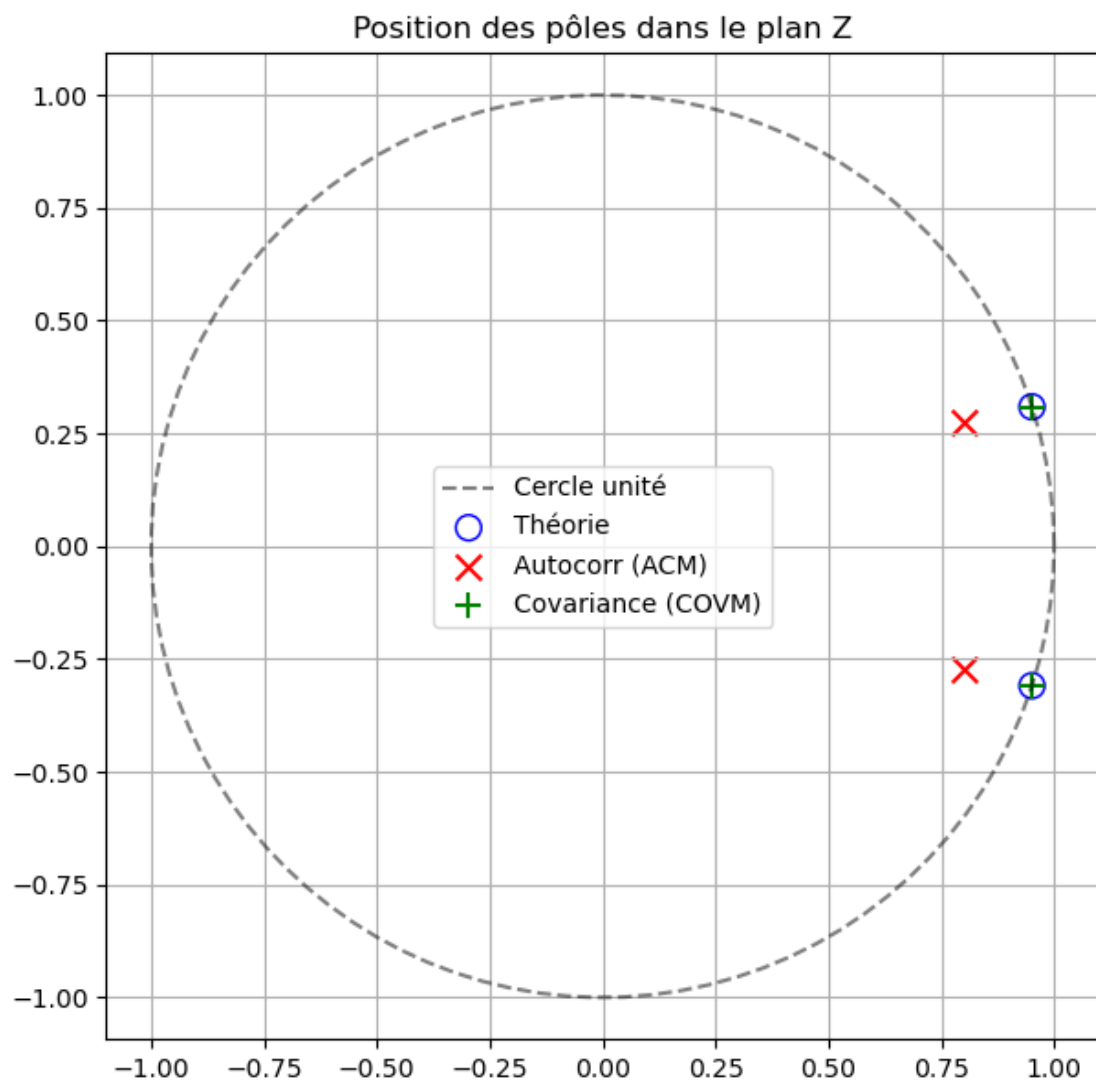
```

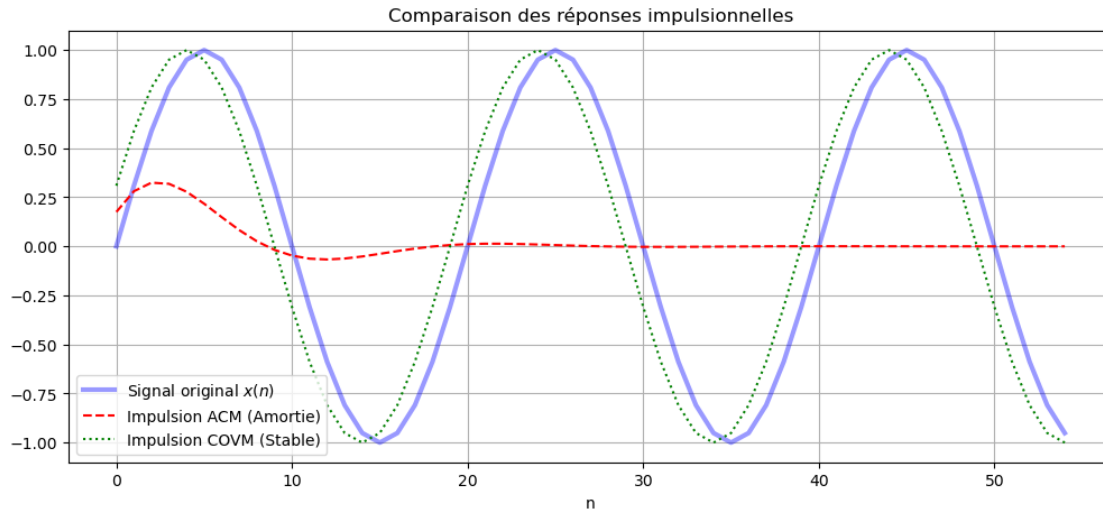
plt.scatter(np.real(poles_cov), np.imag(poles_cov), marker='+', c='g', s=100,
    ↪label="Covariance (COVM)")
plt.title("Position des pôles dans le plan Z")
plt.legend()
plt.grid(True)
plt.axis('equal')
plt.show()

# Réponses Impulsionnelles
h_acm = signal.lfilter(bq0_acm, ap_acm, dn)
h_cov = signal.lfilter([A * np.sin(2*np.pi/N0)], ap_cov, dn) # Gain ajusté pour
    ↪match phase/amp

plt.figure(figsize=(12, 5))
plt.plot(n, x, 'b', label="Signal original  $x(n)$ ", alpha=0.4, linewidth=3)
plt.plot(n, h_acm, 'r--', label="Impulsion ACM (Amortie)")
plt.plot(n, h_cov, 'g:', label="Impulsion COVM (Stable)")
plt.title("Comparaison des réponses impulsionnelles")
plt.xlabel("n")
plt.legend()
plt.grid(True)
plt.show()

```





1.6 5. Analyse du Signal Bruité

Enfin, nous testons la robustesse des méthodes en ajoutant un bruit blanc gaussien au signal. Pour compenser le bruit, il est souvent nécessaire d'augmenter l'ordre du modèle ($p > 2$).

```
[36]: # definition du signal
NO = 20
N= 105
n = np.arange(N)
A=1
x=A * np.sin(2 * np.pi * n / NO)
# Ajout de bruit
sigma_n = 0.2
xn = x + sigma_n * np.random.randn(N)

# impulse response
dn = np.eye(1, N)[0]

# Modélisation avec p=6 pour la robustesse
p_noise = 6
ap_acm_n, bq_acm_n, _ = acm(xn, p_noise)
ap_cov_n, bq_cov_n, _ = covm(xn, p_noise)

#bq_acm_n = np.array([bq_acm_n])
bq_acm_n = np.append(bq_acm_n, np.zeros(p_noise))
bq_cov_n = np.array([ A * np.sin(2*np.pi/NO)])
bq_cov_n = np.append(bq_cov_n, np.zeros(p_noise))

print(f"Coefficients ACM bruité ap : {ap_acm_n}")
```

```

print(f"Gain ACM bruité bq0 : {bq_acm_n}")
print(f"Norme des pôles (ACM bruité) : {np.abs(np.roots(ap_acm_n))}")

print(f"Coefficients COVM bruité ap : {ap_cov_n}")
print(f"Gain COVM bruité bq0 : {bq_cov_n}")
print(f"Norme des pôles (COVM bruité) : {np.abs(np.roots(ap_cov_n))}")

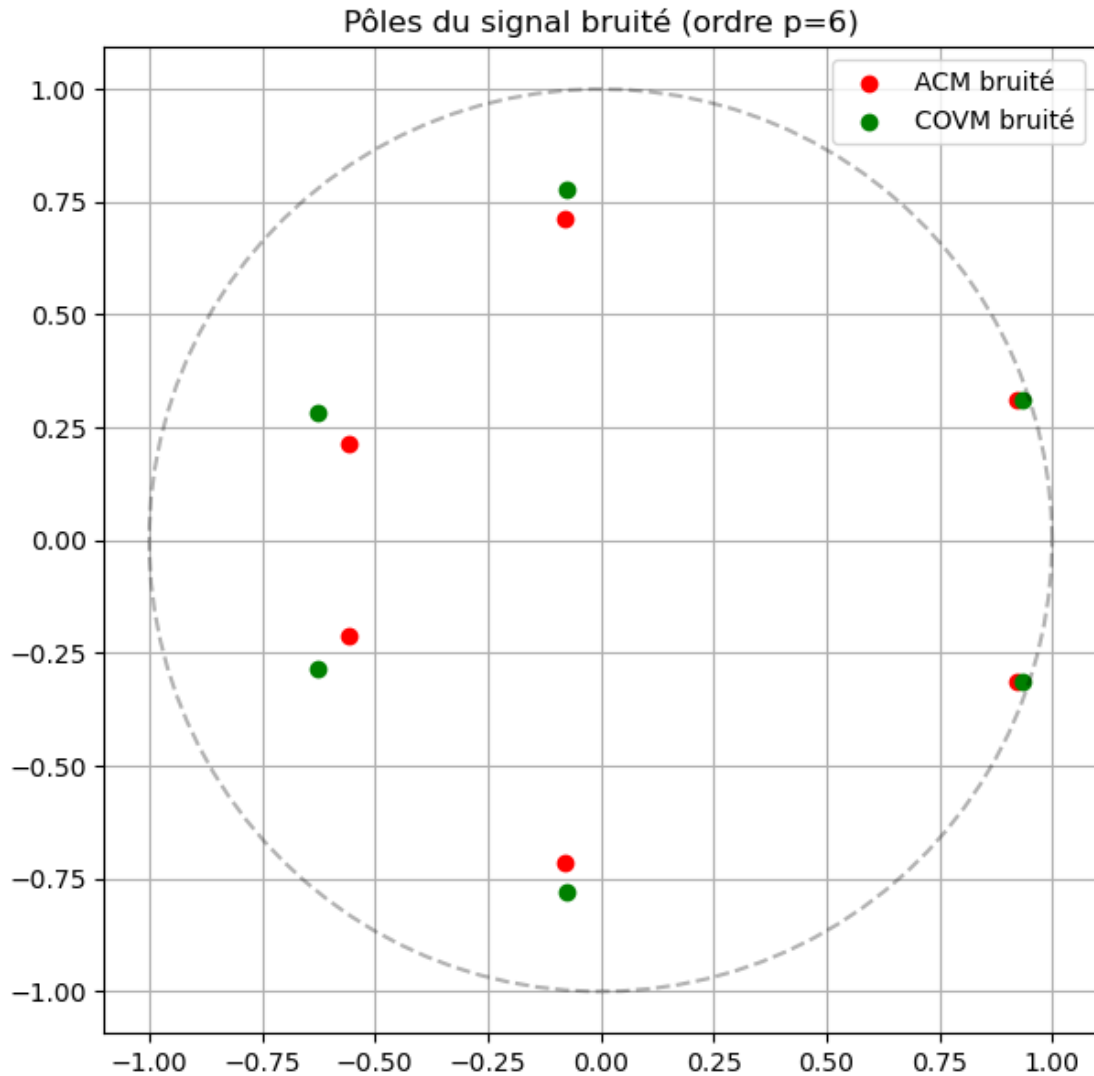
# Visualisation des pôles pour le signal bruité
plt.figure(figsize=(7, 7))
plt.plot(np.cos(theta), np.sin(theta), 'k--', alpha=0.3)
plt.scatter(np.real(np.roots(ap_acm_n)), np.imag(np.roots(ap_acm_n)), c='r',
            ↪label="ACM bruité")
plt.scatter(np.real(np.roots(ap_cov_n)), np.imag(np.roots(ap_cov_n)), c='g',
            ↪label="COVM bruité")
plt.title(f"Pôles du signal bruité (ordre p={p_noise})")
plt.legend()
plt.axis('equal')
plt.grid(True)
plt.show()

```

```

Rx [[ 0.53511345  0.46797057  0.39644055  0.28380991  0.14818188 -0.00522976]
 [ 0.46797057  0.53511345  0.46797057  0.39644055  0.28380991  0.14818188]
 [ 0.39644055  0.46797057  0.53511345  0.46797057  0.39644055  0.28380991]
 [ 0.28380991  0.39644055  0.46797057  0.53511345  0.46797057  0.39644055]
 [ 0.14818188  0.28380991  0.39644055  0.46797057  0.53511345  0.46797057]
 [-0.00522976  0.14818188  0.28380991  0.39644055  0.46797057  0.53511345]]
rx_vec [[ 0.46797057]
 [ 0.39644055]
 [ 0.28380991]
 [ 0.14818188]
 [-0.00522976]
 [-0.14501174]]
Coefficients ACM bruité ap : [ 1.          -0.57058461 -0.35697196 -0.09535011
 0.01559575  0.26115472
 0.17489804]
Gain ACM bruité bq0 : [0.27404404 0.          0.          0.          0.          0.
 0.          ]
Norme des pôles (ACM bruité) : [0.97511411 0.97511411 0.71746278 0.71746278
 0.59777488 0.59777488]
Coefficients COVM bruité ap : [ 1.          -0.46658706 -0.37239293 -0.17919508
-0.04085991  0.2692213
 0.2785519 ]
Gain COVM bruité bq0 : [0.30901699 0.          0.          0.          0.
 0.          0.          ]
Norme des pôles (COVM bruité) : [0.98333432 0.98333432 0.78188668 0.78188668
 0.68644864 0.68644864]

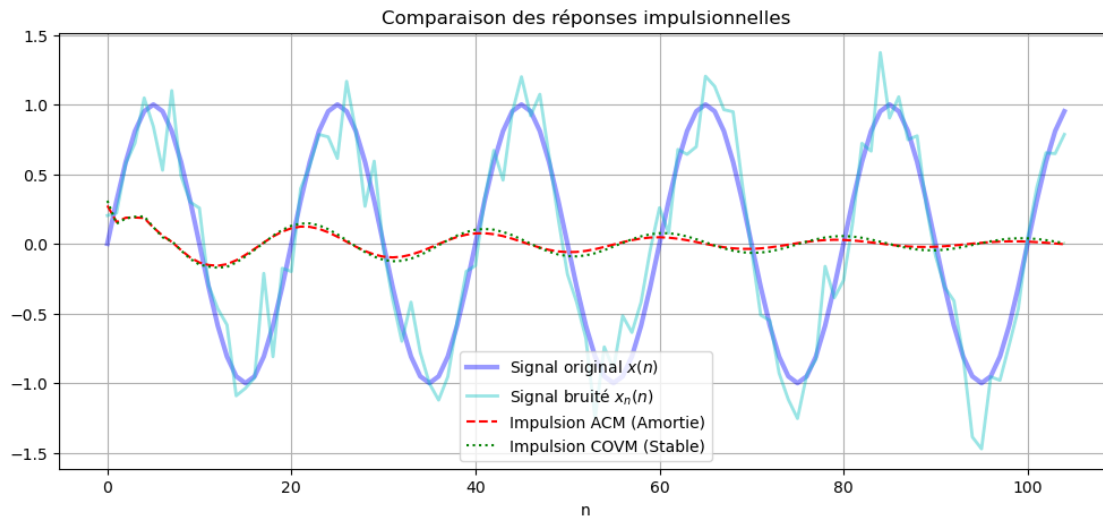
```



```
[37]: # Réponses Impulsionnelles
h_acm_n = signal.lfilter(bq_acm_n, ap_acm_n, dn)
h_cov_n = signal.lfilter(bq_cov_n, ap_cov_n , dn) # Gain ajusté pour match phase/
→ amp

plt.figure(figsize=(12, 5))
plt.plot(n, x, 'b', label="Signal original  $x(n)$ ", alpha=0.4, linewidth=3)
plt.plot(n, xn, 'c', label="Signal bruité  $x_n(n)$ ", alpha=0.4, linewidth=2)
plt.plot(n, h_acm_n, 'r--', label="Impulsion ACM (Amortie)")
plt.plot(n, h_cov_n, 'g:', label="Impulsion COVM (Stable)")
plt.title("Comparaison des réponses impulsionnelles")
plt.xlabel("n")
plt.legend()
```

```
plt.grid(True)
plt.show()
```



1.6.1 L'approche Statistique (Énergie de l'erreur)

Celle qui est utilisé dans les slides du cours. On estime que la puissance du signal d'entrée (l'impulsion) doit compenser l'erreur de prédiction :

$b_0 = \sqrt{|E_p|}$ avec E_p l'erreur de prédiction, calculée

$$E_p = r_x(0) + \sum_{k=1}^p a(k)r_x(k)$$

Limitation : Très sensible au bruit car $r_x(0)$ inclut la puissance du signal et du bruit. Si le signal est faible ou le bruit fort, b_0 peut être largement surestimé, conduisant à une reconstruction de signal avec une amplitude exagérée.

1.6.2 L'approche Déterministe (Matching du premier échantillon)

Comme dans la méthode de Padé, on force le premier échantillon de la réponse impulsionnelle à être égal au premier échantillon du signal observé :

$$b_0 = x(0)$$

Avantage : L'amplitude de départ est toujours correcte. Inconvénient : Si $x(0)$ est très bruité, tout le signal reconstruit sera décalé en amplitude.

1.6.3 L'approche Globale (Méthode de Shanks)

C'est la méthode la plus robuste. Une fois les pôles (coefficients a_p) trouvés, on cherche le b_0 qui minimise l'erreur quadratique sur tous les échantillons par moindres carrés.

$$b = (G^T G)^{-1} G^T x$$

Où G est la réponse impulsionnelle du filtre $1/A(z)$.

```
[38]: # Comparaison des calculs de gain b0 avec bruit et p=6

# 1. Gain via l'erreur de prédiction (Statistique)
#  $E_p = r(0)*a(0) + r(1)*a(1) + \dots$ 
r_long = np.correlate(xn, xn, mode='full') / len(xn)
r_vals = r_long[len(xn)-1 : len(xn)+p_noise]
ep_stat = np.dot(ap_cov_n, r_vals[:p_noise+1])
bq0_stat = np.sqrt(np.abs(ep_stat))

# 2. Gain via le premier échantillon (Déterministe)
bq0_first = xn[0]

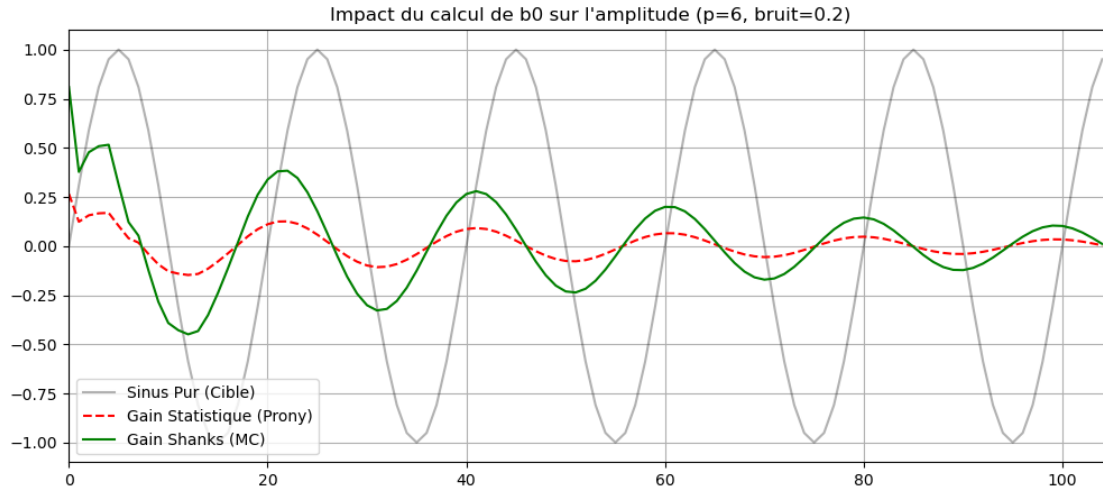
# 3. Gain via Shanks (Moindres Carrés - 1 seul coefficient b0)
# On génère la réponse de  $1/A(z)$ 
gn = signal.lfilter([1.0], ap_cov_n, dn)
#  $b0 = (g^T * x) / (g^T * g)$ 
bq0_shanks = np.dot(gn, xn) / np.dot(gn, gn)

print(f"Gain Statistique : {bq0_stat:.4f}")
print(f"Gain Premier Point : {bq0_first:.4f}")
print(f"Gain Shanks (Optimal) : {bq0_shanks:.4f}")

# Visualisation
h_stat = signal.lfilter([bq0_stat], ap_cov_n, dn)
h_shanks = signal.lfilter([bq0_shanks], ap_cov_n, dn)

plt.figure(figsize=(12, 5))
plt.plot(n, x, 'k', label="Sinus Pur (Cible)", alpha=0.3)
plt.plot(n, h_stat, 'r--', label="Gain Statistique (Prony)")
plt.plot(n, h_shanks, 'g', label="Gain Shanks (MC)")
plt.title(f"Impact du calcul de b0 sur l'amplitude (p={p_noise},  $\sigma_n$  →bruit={sigma_n})")
plt.xlim([0, N])
plt.legend()
plt.grid()
plt.show()
```

```
Gain Statistique : 0.2654
Gain Premier Point : 0.2030
Gain Shanks (Optimal) : 0.8092
```



1.7 Solution : La méthode de “Shanks-Covariance”

Pour bénéficier de la précision de la fréquence (via COVM) tout en optimisant l’amplitude sur tout le record (via Shanks), il est préférable de modifier la fonction pour qu’elle utilise covm par défaut.

En autorisant $q=1$: 1. b_0 et b_1 vont s’ajuster pour créer une interférence qui “décale” la sinusoïde.

2. Le modèle pourra enfin démarrer à 0 et atteindre son pic d’amplitude 1 à la bonne fréquence.

3. L’erreur E_{ls} va chuter drastiquement par rapport à ton gain unique de 1.0470.

```
[44]: from scipy import linalg
from scipy import signal

def shanks_covm(x, p, q):
    """
    Version améliorée de Shanks utilisant la méthode de Covariance
    pour une estimation précise des pôles (ap).
    """
    # 1. Estimation des pôles (ap) via Covariance (plus précis pour sinus)
    N_sig = len(x)
    # Matrice de données sans padding (Covariance)
    X_mat = toeplitz(x[p-1:N_sig-1], x[p-1::-1])
    x_target = x[p:N_sig]

    # Résolution des coefficients ap
    ap_tail = -solve(X_mat.T @ X_mat, X_mat.T @ x_target)
    ap = np.concatenate(([1.0], ap_tail))

    # 2. Estimation du numérateur (bq) sur tout le signal (Shanks)
    # On génère la réponse impulsionnelle du filtre 1/A(z)
    impulse = np.zeros(N_sig)
```

```

impulse[0] = 1.0
gn = signal.lfilter([1.0], ap, impulse)

# Matrice de base pour les zéros
G = linalg.toeplitz(gn, np.zeros(q + 1))

# Moindres carrés pour bq :  $b = (G^T * G)^{-1} * G^T * x$ 
bq = linalg.pinv(G) @ x

xhat = G @ bq
epq = np.linalg.norm(x - xhat)

return ap, bq, epq, xhat

# Test avec votre modèle p=6, q=1
ap_final, bq_final, err_final, xhat_final = shanks_covm(xn, 8, 4)

print(f"Norme des pôles avec Covariance : {np.abs(np.roots(ap_final))}")
print(f"Coefficients bq (q=1) : {bq_final}")

plt.figure(figsize=(12, 5))
plt.plot(n, x, 'k', label="Sinus Pur (Cible)", alpha=0.3)
plt.plot(n, xn, 'c', label="Signal bruité", alpha=0.4)
plt.plot(n, xhat_final, 'g', label="Gain Shanks (MC)")
plt.title(f"Impact du calcul de b0 sur l'amplitude (p={p_noise},  
↳bruit={sigma_n})")
plt.xlim([0, N])
plt.legend()
plt.grid()
plt.show()

# Visualisation des pôles pour le signal bruité
plt.figure(figsize=(7, 7))
plt.plot(np.cos(theta), np.sin(theta), 'k--', alpha=0.3)
plt.scatter(np.real(np.roots(ap_final)), np.imag(np.roots(ap_final)), c='m',  
↳label="Shanks-Covariance")
plt.title(f"Pôles du signal bruité (Shanks-Covariance)")
plt.legend()
plt.axis('equal')
plt.grid(True)
plt.show()

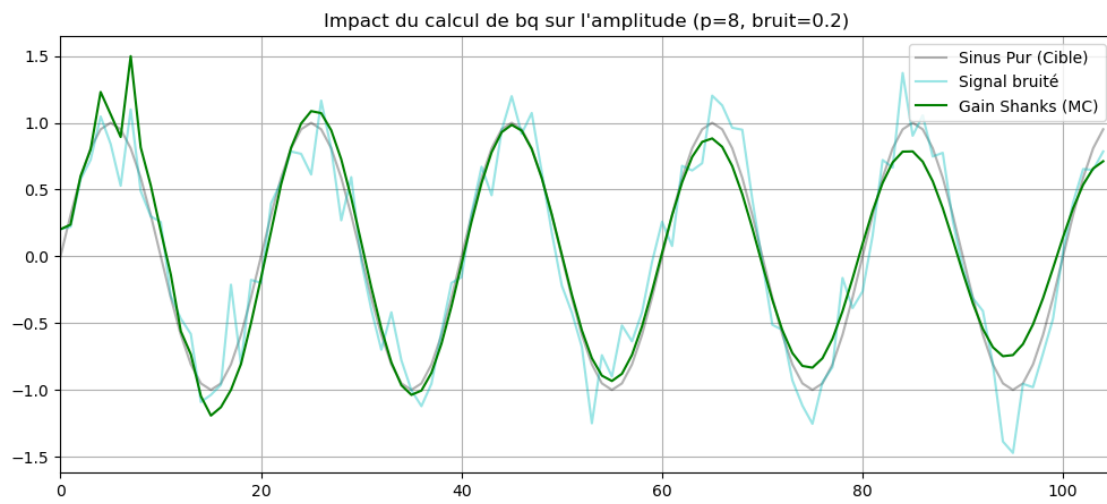
```

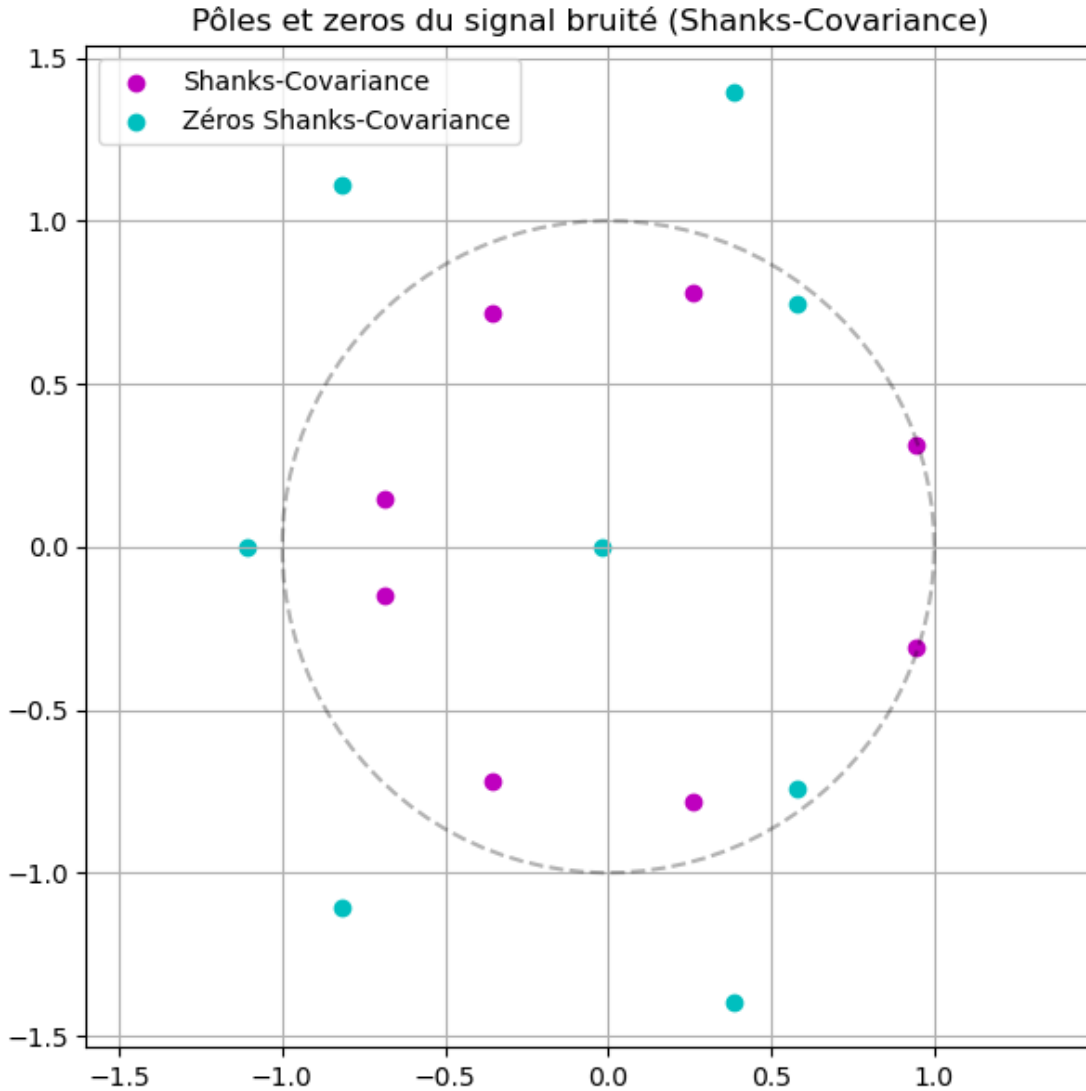
```

Norme des pôles avec Covariance : [0.99458879 0.99458879 0.82330928 0.82330928
0.80192294 0.80192294
0.70053104 0.70053104]
Coefficients bq (q=1) : [-0.49931411 -0.04263188 0.46764383 0.94416517

```

1.48845608]





1.8 6.Remarques sur les résultats :

ACM : On observe que les pôles sont tirés vers le centre. Cela est dû au fait que la fenêtre finie est vue comme une attaque et une extinction du signal (fenêtrage rectangulaire implicite).

COVM : Les pôles restent beaucoup plus proches du cercle unité. C'est la méthode à privilégier pour l'estimation de fréquences pures sur des segments courts.

Bruit : En augmentant l'ordre à $p=6$, le modèle parvient à mieux isoler la composante sinusoïdale au milieu du spectre de bruit.